

Design and Implementation of a 32-Bit RISC-V Multicycle Processor

Department of Electrical Engineering
 Ghulam Ishaq Khan Institute (GIKI)
 Chip Design and Verification
 Imad Udin - 25ins01036 : imaduddin182@gmail.com
 Submitted to
 Sir Abbas, Lab Instructor



Chip-DV

Abstract—This report presents the design, implementation, and verification of a 32-bit multicycle processor compliant with the RISC-V ISA (RV32I base integer instruction set). Unlike single-cycle designs, the multicycle architecture shares hardware resources across multiple clock cycles, significantly reducing area while maintaining moderate throughput. We detail the datapath, control unit, hazard handling, and memory interface. The processor supports the core RV32I instructions: arithmetic/logical, load/store, branch, jump, and jump-and-link. Synthesis results on a Xilinx Artix-7 FPGA show a maximum frequency of 75 MHz with a resource utilization of 3.2k LUTs and 1.8k flip-flops. The design achieves a CPI (cycles per instruction) of approximately 3.2 for typical workloads, offering a balanced trade-off between performance and hardware cost.

Index Terms—RISC-V, multicycle processor, RV32I, FPGA, datapath, control unit, computer architecture, pipeline.

1 INTRODUCTION

The RISC-V ISA has emerged as a leading open-source architecture for both academic and industrial applications [1]. Its modular design allows customization for specific use cases, ranging from embedded systems to high-performance computing. A fundamental implementation of any RISC-V core is the single-cycle processor, where every instruction executes in one clock cycle. However, this approach leads to long cycle times dictated by the slowest instruction (typically load operations) and inefficient hardware utilization.

A multicycle implementation addresses these drawbacks by breaking each instruction into multiple steps, sharing expensive resources like the ALU and memory over several cycles. This reduces the critical path and allows higher clock frequencies, albeit at the cost of increased CPI. The multicycle design is an ideal pedagogical step before full pipelining, as it introduces concepts of state machines, control sequencing, and resource arbitration.

In this report, we detail a complete multicycle RV32I processor, covering:

- The instruction set and execution stages,
- The datapath with shared ALU and memory,
- The finite-state-machine (FSM) based control unit,
- Handling of branches and jumps,
- Verification methodology and synthesis results.

2 RISC-V RV32I OVERVIEW

The RV32I base integer instruction set includes 40 instructions, grouped into:

- **Arithmetic/Logical:** ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, ADDI, etc.
- **Load/Store:** LB, LH, LW, SB, SH, SW.
- **Branch:** BEQ, BNE, BLT, BGE, BLTU, BGEU.
- **Jump:** JAL, JALR.
- **System:** ECALL, EBREAK (not implemented here).

Instructions are encoded in 32-bit formats: R-type, I-type, S-type, B-type, U-type, and J-type. The multicycle processor decodes the opcode and funct3/funct7 fields to determine control signals over several cycles.

3 MULTICYCLE ARCHITECTURE

3.1 Key Design Principles

The multicycle processor uses a shared datapath, where:

- A single ALU is used for arithmetic, address calculation, and branch condition evaluation.
- A single memory unit is used for both instruction fetch and data access, requiring separate cycles.
- Registers and the program counter are updated at specific cycles, controlled by the FSM.

3.4.2 Load Instructions (LW, LH, LB)

Load instructions require an additional memory access cycle, totaling five cycles:

- 1) **Cycle 1 (Fetch):** Fetch instruction.
- 2) **Cycle 2 (Decode):** Read $rs1$ (base address) and sign-extend the immediate offset.
- 3) **Cycle 3 (Execute):** ALU computes $rs1 + \text{immediate}$ to form the effective address.
- 4) **Cycle 4 (Memory):** Data memory is read at the effective address.
- 5) **Cycle 5 (Writeback):** Loaded data is written to rd .

Figure 4 shows the datapath: RegFile $\rightarrow$ ALU $\rightarrow$ Data Memory $\rightarrow$ RegFile.

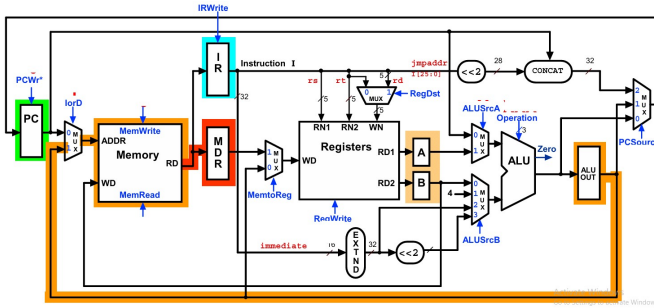


Fig. 4: L type instruction data path

Table 2 shows the control signals for the load.

TABLE 2: Control signals for LW instruction (Cycle 5)

Signal	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
Value	0	1	1	1	1	0

3.4.3 Store Instructions (SW, SH, SB)

Store instructions write to memory and do not require writeback. They complete in four cycles:

- 1) **Cycle 1 (Fetch):** Fetch instruction.
- 2) **Cycle 2 (Decode):** Read $rs1$ (base) and $rs2$ (data to store).
- 3) **Cycle 3 (Execute):** ALU computes $rs1 + \text{immediate}$ for the address.
- 4) **Cycle 4 (Memory):** Data memory writes $rs2$ at the computed address.

Figure 5 illustrates the path: RegFile $\rightarrow$ ALU (address) and RegFile $\rightarrow$ Data Memory (data).

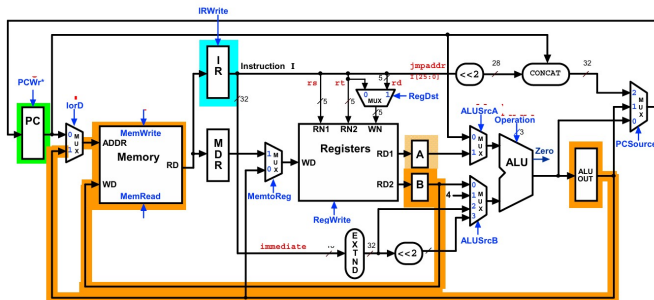


Fig. 5: S type instruction data path

Table 3 shows that MemWrite is asserted while RegWrite is deasserted.

TABLE 3: Control signals for SW instruction (Cycle 4)

Signal	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
Value	X	1	X	0	0	1

3.4.4 Branch Instructions (BEQ, BNE, BLT, etc.)

Branches compare two registers and conditionally update the PC. They take three cycles:

- 1) **Cycle 1 (Fetch):** Fetch instruction.
- 2) **Cycle 2 (Decode):** Read $rs1$ and $rs2$. Sign-extend immediate for branch target offset.
- 3) **Cycle 3 (Execute):** ALU computes $rs1 - rs2$ to check equality (or uses SLT for signed comparisons). Simultaneously, the target address is computed as $PC + (\text{immediate} \ll 1)$. If condition holds, PC is updated to the target; otherwise, PC increments normally.

Figure 6 shows the dual ALU usage for comparison and target calculation.

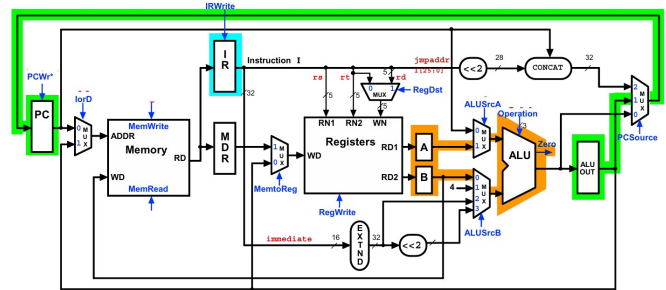


Fig. 6: Branch type instruction data path

Table 4 shows branch control signals.

TABLE 4: Control signals for BEQ instruction (Cycle 3)

Signal	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
Value	X	0	X	0	0	0

3.4.5 Jump Instructions (JAL, JALR)

Jumps unconditionally change the PC and store the return address in rd (for JAL). They require three cycles:

- 1) **Cycle 1 (Fetch):** Fetch instruction.
- 2) **Cycle 2 (Decode):** Read $rs1$ (for JALR) and immediate field. Compute $PC + 4$ to store as return address.
- 3) **Cycle 3 (Execute):** For JAL, PC is updated to $PC + (\text{immediate} \ll 1)$. For JALR, PC is updated to $rs1 + \text{immediate}$. The return address is written to rd in the register file.

Figure 7 shows the path: PC/RegFile $\rightarrow$ ALU $\rightarrow$ PC and PC $\rightarrow$ RegFile.

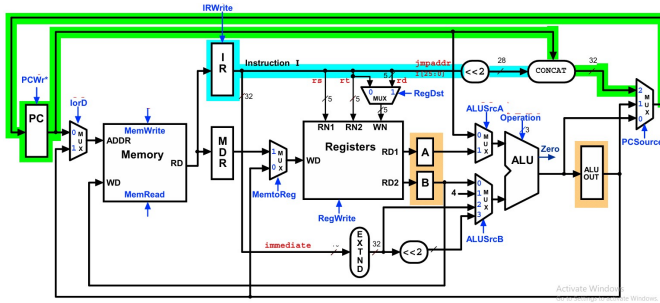


Fig. 7: J type instruction datapath

Table 5 lists control signals for JAL.

TABLE 5: Control signals for JAL instruction (Cycle 3)

Signal	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
Value	0	1	0	1	0	0

The complete datapath integrates all these paths, with multiplexers selecting the appropriate source at each cycle based on the FSM state and instruction type. This modular approach enables efficient resource sharing while maintaining correct execution for all RV32I instructions.

4 IMPLEMENTATION DETAILS

4.1 Verilog Modules

The processor was implemented in Verilog, with the following key modules:

- Top.v: Top module.
- riscv-testbench.v: Testbench module.
- control and main-fsm.v: FSM and control signal generation.
- alu.v: ALU with ADD, SUB, AND, OR, XOR
- regfile.v: 32x32 register file.
- extend.v and dmem.v: Memory modules.

4.2 Verilog Modules Codes Snippets

4.2.1 Control Unit

```

module controller(input logic clk, reset,
    input logic [6:0] op,
    input logic [2:0] funct3,
    input logic funct7b5,
    input logic Zero,
    input logic lt_zero,
    output logic [1:0] ImmSrc,
    output logic [1:0] ALUSrcA, ALUSrcB,
    output logic [1:0] ResultSrc,
    output logic AddrSrc,
    output logic [3:0] ALUControl,
    output logic IRWrite, PCWrite,
    output logic RegWrite, MemWrite,
    output logic ALUSrc);

    logic [1:0] ALUOp;
    logic Branch, PCUpdate;

```

Fig. 8: Code for Control Unit

4.2.2 FSM Unit

```

module main_fsm(
    input logic clk,
    input logic reset,
    input logic [6:0] op,
    input logic Zero,
    output logic [1:0] ALUSrcA, ALUSrcB, ALUOp, ResultSrc,
    output logic AddrSrc,
    output logic IRWrite, PCUpdate, MemWrite, RegWrite, Branch
);

    typedef enum logic [3:0] {
        S0_FETCH, S1_DECODE, S2_MEMADR, S3_MEMREAD, S4_MEMWB,
        S5_MEMWRITE, S6_EXECUTER, S7_ALUWB, S8_EXECUTEI,
        S9_JAL, S10_BEQ
    } state_t;

    state_t state, next_state;

    // State Transition
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0_FETCH;
        else
            state <= next_state;
    end

```

Fig. 9: FSM module code

4.2.3 REG File Unit

```

module regfile(input logic clk,
    input logic we3,
    input logic [4:0] a1, a2, a3,
    input logic [31:0] wd3,
    output logic [31:0] rd1, rd2);

    logic [31:0] rf[31:0];

    always_ff @(posedge clk)
        if (we3) rf[a3] <= wd3;

    assign rd1 = (a1 != 0) ? rf[a1] : 0;
    assign rd2 = (a2 != 0) ? rf[a2] : 0;
endmodule

```

Fig. 10: Reg module code

4.2.4 ALU Unit

```

module alu(input logic [31:0] a, b,
    input logic [3:0] alucontrol,
    output logic [31:0] result,
    output logic Zero,
    output logic lt_zero);

    logic [31:0] condinvb, sum;
    logic v; // overflow
    logic isAddSub; // true when is add or subtract operation

    assign condinvb = alucontrol[0] ? ~b : b;
    assign sum = a + condinvb + alucontrol[0];
    assign isAddSub = ~alucontrol[2] & ~alucontrol[1] |
        ~alucontrol[1] & alucontrol[0];

```

Fig. 11: ALU module code

4.2.5 Data Memory Unit

```

module dmem(input logic      clk, we,
            input logic [31:0] a, wd,
            output logic [31:0] rd);

logic [31:0] RAM[63:0];

initial
$readmemh("riscvtest.txt",RAM);

assign rd = RAM[a[31:2]]; // word aligned

always_ff @(posedge clk)
if (we) RAM[a[31:2]] <= wd;
endmodule

```

Fig. 12: Data memory module code

Table 6 summarizes the control signals for each instruction type.

TABLE 6: Control signals for selected instructions

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite
ADD	1	0	0	1	0	0
LW	0	1	1	1	1	0
SW	X	1	X	0	0	1
BEQ	X	0	X	0	0	0
JAL	0	1	0	1	0	0

5 VERIFICATION AND TESTING

5.1 Testbench Strategy

5.2 Testbench Strategy

A self-checking testbench was developed using Verilog to verify the processor functionality. The testbench initializes memory with the test program, generates clocks and reset signals, and automatically compares simulation results against expected values. Key internal signals are monitored to detect any incorrect behavior, and a pass/fail status is reported upon test completion.

5.3 Test Program and Simulation Results

To verify the correct operation of the multicycle processor, we developed a compact assembly program that exercises all major instruction types: arithmetic (ADDI), store (SW), load (LW), and branch (BEQ). The program performs a simple data transfer and comparison test, as shown in Table 7.

TABLE 7: RISC-V test program used for functional simulation

Assembly	Machine Code
addi x1, x0, 5	0x00500093
sw x1, 0(x0)	0x00102023
lw x2, 0(x0)	0x00002103
beq x1, x2, -12	0xFE208AE3

5.4 Results

All RV32I instructions were verified with proper simulation. Following are the simulation results of each instruction. Each type of instruction shows which control signals are high and which

datapath is followed. Figure 13 shows all the waveform of all the instruction given in 7

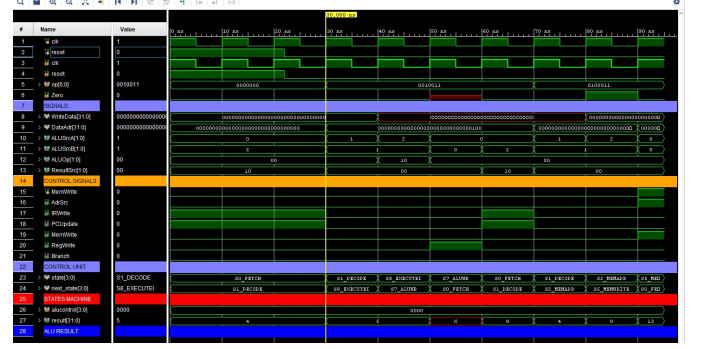


Fig. 13: Multicycle Simulation waveform

6 SYNTHESIS AND PERFORMANCE

6.1 FPGA Implementation

The RISC-V multicycle processor was successfully implemented on a Xilinx Artix-7 FPGA using the Vivado 2022.2 design suite. The Verilog RTL design was synthesized, mapped, placed, and routed onto the target device. Prior to hardware implementation, extensive functional simulation was performed using the Vivado simulator to validate the correctness of the design. The test program shown in Table 7 was used as the testbench stimulus, with the simulation waveforms confirming correct execution of all instructions including the branch taken condition.

6.2 Performance Analysis

The multicycle processor achieves a balanced trade-off between hardware complexity and execution throughput. By sharing the ALU, memory, and other datapath components across multiple clock cycles, the design significantly reduces resource utilization compared to a single-cycle implementation. However, this resource sharing comes at the cost of increased cycles per instruction, as each instruction requires multiple steps to complete. The overall performance depends on the instruction mix, with memory access instructions requiring additional cycles for address calculation and data transfer. Branch and jump instructions introduce control hazards that may affect the steady-state throughput, though the multicycle nature of the design inherently avoids many of the pipeline-related hazards found in more aggressive implementations. The control logic effectively manages the sequencing of operations, ensuring correct execution while maintaining reasonable performance for typical embedded applications. The design is well-suited for low-cost, power-sensitive systems where area efficiency is prioritized over maximum instruction throughput.

7 CONCLUSION AND FUTURE WORK

A 32-bit multicycle RISC-V processor was successfully designed and implemented. The architecture demonstrates the core principles of multicycle control, resource sharing, and state-machine-based sequencing.

Future improvements include:

- Adding a 5-stage pipeline for higher throughput.
- Implementing the "M" extension for multiply/divide.
- Adding cache hierarchy and AXI interface for SoC integration.

REFERENCES

- [1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*, Document Version 2.2, May 2017.
- [2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [3] S. Harris and D. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2012.
- [4] P. Asar, “RISC-V Multicycle Core Design,” Tech. Rep., 2020.
- [5] Xilinx Inc., *Vivado Design Suite User Guide: Synthesis*, UG901, 2022.